ВНУТРІШНІЙ ДОКУМЕНТ

FICE SELECTION COMMITTEE

Повний звіт про релізи

Версії 1.0.0 – 1.2.0

FICE-REL-2026-003

Охоплений період: 25 – 30 березня 2026 р.

Поточна версія: v1.2.0

Дата документа: 30 березня 2026 р.

Підготовлено: Release Engineering Team

Статус документа: Затверджено

Класифікація: Внутрішній

Версія документа: 4.0

Зміст

Частина А: Нетехнічний огляд

А.1 Вступ

А.2 Загальний огляд платформи

А.3 Що нового у кожній версії

- А.3.1 v1.0.0 — Фундамент
- А.3.2 v1.1.0 — Інфраструктура
- А.3.3 v1.2.0 — Зрілість

А.4 Нові можливості для користувачів

А.5 Хронологія та статистика

А.6 Подальші плани

Частина Б: Технічна документація

Б.1 Визначення та скорочення

Б.2 Організація GitHub — повний перелік репозиторіїв

Б.3 Технічний стек

Б.4 Хронологія релізів

Б.5 Версія 1.0.0 — Фундамент проєкту

- Б.5.1 Цілі та обсяг
- Б.5.2 Перелік змін
- Б.5.3 Архітектура монорепозиторію
- Б.5.4 Спільні бібліотеки

Б.6 Версія 1.1.0 — Інфраструктура та тестування

- Б.6.1 Цілі та обсяг
- Б.6.2 Перелік змін
- Б.6.3 Docker Compose та сервіси
- Б.6.4 CI/CD конвеєр

Б.6.5 Тестова інфраструктура
Б.6.6 OpenAPI та фронтенд-інтеграція
Б.7 Версія 1.2.0 — Зрілість Docker та розширення
Б.7.1 Цілі та обсяг
Б.7.2 Перелік змін
Б.7.3 Багатоетапні Docker-збірки
Б.7.4 Виправлення Auth API (POST → GET)
Б.7.5 Docker Compose: повна переробка
Б.7.6 CI/CD: розширення та сканування вразливостей
Б.7.7 Telegram Bot: новий сервіс
Б.7.8 Моніторинг та спостережуваність
Б.7.9 Інженерна документація
Б.8 Frontend — selection-committee-web
Б.8.1 Огляд
Б.8.2 Перелік змін
Б.8.3 Ключові функціональності
Б.9 Ламаючі зміни та міграція
Б.10 Поточна архітектура системи
Б.11 Як запускати / деплоїти / моніторити / тестувати
Б.12 Детальне порівняння «Було → Стало»
Б.13 Ризики та подальші кроки
Б.14 Контроль документа
А. Додаток А: Повний журнал комітів

ЧАСТИНА A

Нетехнічний огляд

Загальний опис платформи, нові можливості для користувачів, хронологія розвитку та плани на майбутнє — без технічної термінології.

A.1 Вступ

Цей документ описує еволюцію платформи FICE Selection Committee від створення до версії 1.2.0. Він розділений на дві частини: Частина А містить загальний огляд зрозумілою мовою без технічних деталей, Частина Б — повну технічну документацію для інженерної команди.

Платформа розроблялася протягом 6 днів (25–30 березня 2026 р.) і пройшла шлях від порожнього проєкту до повнофункціональної системи з автоматизованою збіркою, моніторингом, тестуванням та інженерними стандартами.

A.2 Загальний огляд платформи

FICE Selection Committee — платформа для управління процесом вступу до навчальних закладів. Система автоматизує ключові процеси приймальної комісії та забезпечує зручну взаємодію з абітурієнтами.

Основні функції платформи:

- **Автентифікація користувачів** — безпечний вхід через одноразові email-посилання без необхідності запам'ятовувати паролі.
- **Управління документами** — завантаження, зберігання та обробка документів абітурієнтів.
- **Обробка заявок на вступ** — прийом та опрацювання заяв абітурієнтів.
- **Сповіщення** — автоматичне інформування користувачів через email.
- **Управління функціональністю** — можливість вмикати та вимикати окремі функції системи через месенджер, без необхідності зупиняти роботу платформи.
- **Моніторинг** — автоматичне відстеження стану всіх компонентів системи з візуальними панелями.

A.3 Що нового у кожній версії

A.3.1 Версія 1.0.0 (25–26 березня): Фундамент

Перша версія заклала основу проєкту. Розрізнені компоненти системи, що існували окремо, були об'єднані в єдиний проєкт із спільною структурою. Створено перевикористовувані компоненти, щоб уникнути дублювання роботи між різними частинами системи. Налаштовано механізм публікації готових компонентів для подальшого використання.

A.3.2 Версія 1.1.0 (26–28 березня): Інфраструктура

Друга версія додала операційну інфраструктуру. З'явилося автоматичне тестування, що перевіряє коректність роботи системи при кожній зміні. Створено панелі моніторингу для відстеження здоров'я системи у реальному часі. Налаштовано повне локальне середовище розробки, де весь стек можна запустити однією командою. Впроваджено автоматичний конвеєр збірки та публікації.

A.3.3 Версія 1.2.0 (30 березня): Зрілість

Третя версія сфокусована на зрілості та безпеці. Додано автоматичне сканування на вразливості при кожній збірці. Створено бот у месенджері Telegram для оперативного управління системою. Повністю перероблено зовнішній вигляд платформи з новою системою входу через email-посилання (без паролів). Додано юридичні сторінки. Покращено процес контейнеризації для зменшення розміру та підвищення безпеки.

A.4 Нові можливості для користувачів

- **Вхід через email-посилання** — замість запам'ятовування пароля користувач отримує одноразове посилання на електронну пошту. Один клік — і ви в системі.
- **Оновлений дизайн головної сторінки** — нова панельна розкладка з поділом екрану на інформаційну та функціональну частини.
- **Триетапний онбордінг** — покроковий процес реєстрації з обов'язковим ознайомленням із юридичними документами (правилами використання та угодою про конфіденційність).
- **Панель управління (Dashboard)** — персональний кабінет користувача з доступом до основних функцій.
- **Юридичні сторінки** — правила використання платформи та угода про конфіденційність, доступні для перегляду у будь-який час.
- **Управління функціональністю через Telegram** — адміністратори можуть вмикати та вимикати функції платформи безпосередньо з месенджера Telegram.
- **Покращена доступність (WCAG)** — інтерфейс адаптовано для людей з обмеженими можливостями: навігація клавіатурою, підтримка програм зчитування з екрану, покращений контраст кольорів.

A.5 Хронологія та статистика

Дата	Подія
25 березня 2026	Створення проєкту, початок об'єднання компонентів
26 березня 2026	Випуск v1.0.0 — фундамент готовий
26–28 березня 2026	Розробка інфраструктури, тестів, моніторингу
28 березня 2026	Випуск v1.1.0 — інфраструктура готова
30 березня 2026	Випуск v1.2.0 — зрілість та безпека

Ключові числа:

- 8 серверних компонентів (включаючи шлюз та Telegram-бот)
- 1 клієнтський застосунок (веб-інтерфейс)
- 13 репозиторіїв у організації GitHub
- 4 панелі моніторингу (автоматично створюються при першому запуску)
- Автоматичне сканування безпеки при кожній збірці
- Повний стек запускається однією командою

A.6 Подальші плани

Розвиток платформи продовжується за кількома напрямками:

- **Продовження оновлення інтерфейсу** — реалізація запланованих покращень дизайну для підвищення зручності використання.
- **Покращення доступності** — подальша адаптація інтерфейсу для людей з обмеженими можливостями відповідно до міжнародних стандартів.
- **Підготовка до розгортання** — створення документації та конфігурацій для запуску платформи у промисловому середовищі.
- **Оптимізація швидкодії** — зменшення часу збірки та прискорення роботи системи.

ЧАСТИНА Б

Технічна документація

Архітектура, технічний стек, деталі кожного релізу, порівняння змін, ризики та інструкції з запуску — для інженерної команди.

Б.1 Визначення та скорочення

Термін / Скорочення	Визначення
Biome	Інструмент для лінтингу та форматування JavaScript/TypeScript коду (заміна ESLint + Prettier)
buildSrc convention plugin	Gradle-плагін у директорії buildSrc/, що визначає спільні конвенції збірки для всіх модулів проєкту
CI/CD	Continuous Integration / Continuous Delivery — безперервна інтеграція та доставка
CRUD	Create, Read, Update, Delete — базові операції над даними
E2E	End-to-End — наскрізне тестування всього потоку від UI до бази даних
Feign	Декларативний HTTP-клієнт для міжсервісної комунікації у Spring Cloud
Feature Flag	Механізм увімкнення/вимкнення функціональності системи без повторного розгортання
Flyway	Інструмент версіонування та міграції схем баз даних
GHCR	GitHub Container Registry — реєстр контейнерних образів GitHub
Grafana	Платформа візуалізації метрик та моніторингу з провізійованими дашбордами
i18n	Internationalization — інтернаціоналізація (підтримка кількох мов)
JDK / JRE	Java Development Kit / Java Runtime Environment
JWT	JSON Web Token — токен автентифікації
Magic Link	Метод автентифікації через одноразове посилання, надіслане на email
MinIO	S3-сумісне об'єктне сховище для зберігання файлів та документів
Multi-stage build	Багатоетапна Docker-збірка, де кожен етап використовує окремий базовий образ для зменшення фінального розміру

Термін / Скорочення	Визначення
ngrok	Інструмент для створення публічного тунелю до локального сервера (використовується для Telegram webhook)
OpenAPI	Специфікація для опису REST API у машинозчитуваному форматі
Playwright	Фреймворк для наскрізного тестування веб-застосунків
Prometheus	Система збору та зберігання часових рядів метрик від мікросервісів
RabbitMQ	Брокер повідомлень для асинхронної комунікації між мікросервісами
REST	Representational State Transfer — архітектурний стиль API
SARIF	Static Analysis Results Interchange Format — формат звітів статичного аналізу
Semver	Semantic Versioning — семантичне версіонування (MAJOR.MINOR.PATCH)
Spring Cloud Contract	Фреймворк для контрактного тестування міжсервісних API (provider та consumer сторони)
TanStack Query	Бібліотека для управління серверним станом у React (кешування, рефетчинг, інвалідація)
Testcontainers	Бібліотека для запуску контейнерних залежностей під час тестування
Trivy	Open-source сканер вразливостей для контейнерних образів від Aqua Security
Zipkin	Система розподіленого трасування для відстеження запитів між мікросервісами
Zustand	Легка бібліотека управління клієнтським станом у React

Б.2 Організація GitHub — повний перелік репозиторіїв

Репозиторій	Опис	Гілка
Монорепозиторій (backend)		
`fice-selection-committee`	Backend: мікросервіси, бібліотеки, інфраструктура	`main`
Оригінальні сервіси (до міграції в монорепозиторій)		
`selection-committee-identity-service`	Автентифікація, JWT, magic link	—
`selection-committee-admission-service`	Заяви абітурієнтів	—
`selection-committee-documents-service`	Зберігання документів, MinIO	—
`selection-committee-environment-service`	Feature flags, конфігурація	—
`selection-committee-notifications-service`	Email-сповіщення	—
`selection-committee-gateway`	API-шлюз	—
Frontend		
`selection-committee-web`	Next.js 16, TypeScript, shadcn/ui	`develop`
Додано у v1.2.0		
`selection-committee-telegram-bot-service`	Telegram-бот для feature flags	—
Інфраструктура та інструменти		
`selection-committee-infra`	Інфраструктурні конфігурації	—
`selection-committee-bootstrap`	Ініціалізація середовища	—

Репозиторій	Опис	Гілка
`selection-committee-e2e-tests`	Playwright E2E тести (мігровано з монорепо у v1.2.0)	—
`selection-committee-computer-vision`	Комп'ютерний зір	—

Б.3 Технічний стек

Параметр	Значення
Організація	fice-selection-committee (GitHub)
Backend структура	Gradle multi-module монорепозиторій
Мова серверної частини	Java 21 (Spring Boot 3.x, Spring Cloud)
Мова клієнтської частини	TypeScript (Next.js 16+, React, Tailwind v4, shadcn/ui)
Система збірки (backend)	Gradle 8.14, buildSrc convention plugins
Система збірки (frontend)	pnpm, Next.js 16, Biome, Vitest, Playwright
Інфраструктура	Docker Compose, PostgreSQL 16, Redis 7, RabbitMQ 3, MinIO, Mailpit, Zipkin, Prometheus, Grafana

Б.4 Хронологія релізів

Версія	Дати	Тема	Комітів	Файлів	Рядків (+/−)
v1.0.0	25–26.03.2026	Фундамент: монорепозиторій, міграція сервісів, спільні бібліотеки	9	67	+17 080
v1.1.0	26–28.03.2026	Інфраструктура: Docker Compose, CI/CD, тестування, OpenAPI	18	55	+7 817
v1.2.0	30.03.2026	Зрілість: multi-stage Docker, Telegram Bot, Trivy, документація	12	90	+14 354 / −5 309
Всього	**25–30.03.2026**	**—**	**39**	**—**	**—**

> Усі три релізи відбулися протягом 6 днів. Проєкт рухався від порожнього репозиторію до повнофункціональної мікросервісної платформи з CI/CD, моніторингом, контрактними тестами та інженерними стандартами.

Б.5 Версія 1.0.0 — Фундамент проєкту (25–26 березня 2026)

Б.5.1 Цілі та обсяг

Перший реліз створив фундамент платформи: Gradle multi-module монорепозиторій, міграція 6 існуючих мікросервісів, розробка спільних бібліотек та налаштування публікації до GitHub Packages.

Ключові цілі:

- Об'єднати розрізнені репозиторії сервісів у єдиний монорепозиторій з централізованою збіркою.
- Створити спільні бібліотеки для уникнення дублювання коду між сервісами.
- Встановити інфраструктуру публікації артефактів (GitHub Packages).
- Забезпечити скрипти локального розгортання та стартового набору розробника.

Б.5.2 Перелік змін

№	Ref	Категорія	Опис
1	`71b7789`	[НОВИЙ]	Gradle multi-module монорепозиторій: інфраструктура збірки, buildSrc convention plugins
2	`8a8c205`	[ВИПРАВЛЕНО]	Додано execute permission для gradlew
3	`6db6b73`	[НОВИЙ]	Злиття sc-auth-starter у монорепозиторій як спільну бібліотеку
4	`046d46b`	[ВИПРАВЛЕНО]	Додано client package для sc-auth-starter, виправлено scope у .gitignore
5	`6701695`	[ВИПРАВЛЕНО]	Unauthorized entry point, нормалізація role authority names
6	`999dc4c`	[ІНШЕ]	Скрипт локального запуску та логотип проєкту
7	`d6f74b1`	[НОВИЙ]	Реалізація спільних бібліотек: sc-test-common, sc-observability-starter, sc-s3-starter
8	`bdde5d4`	[ВИПРАВЛЕНО]	Реєстрація S3BucketInitializer як bean у auto-configuration sc-s3-starter
9	`30f2b86`	[НОВИЙ]	Публікація спільних бібліотек v1.0.0 до GitHub Packages

Б.5.3 Архітектура монорепозиторію

Версія 1.0.0 встановила структуру монорепозиторію, що об'єднує усі компоненти платформи:

Директорія	Призначення
`server/services/`	6 мікросервісів: identity, admission, documents, environment, notifications, gateway
`server/libs/`	Спільні бібліотеки для перевикористання між сервісами
`server/buildSrc/`	Convention plugins: sc.java-conventions, sc.spring-boot-service, sc.testing-conventions, sc.jacoco-conventions
`client/web/`	Next.js фронтенд
`infra/`	Docker Compose, конфігурація інфраструктури

Б.5.4 Спільні бібліотеки

Бібліотека	Призначення	Споживачі
`sc-auth-starter`	JWT автентифікація, Spring Security фільтри, Feign-інтерсептори	Усі сервіси
`sc-test-common`	Базові класи для інтеграційних тестів, Testcontainers конфігурація	Усі сервіси (test scope)
`sc-observability-starter`	Tracing, metrics, Spring Boot Actuator конфігурація	Усі сервіси
`sc-s3-starter`	MinIO/S3 клієнт, автоматична ініціалізація бакетів	documents-service

> **Принцип:** зміна у будь-якій спільній бібліотеці вимагає виконання `./gradlew publishToMavenLocal` з кореня `server/` перед збіркою залежних сервісів. Це обов'язкова умова ланцюжка збірки.

Б.6 Версія 1.1.0 — Інфраструктура та тестування (26–28 березня 2026)

Б.6.1 Цілі та обсяг

Другий реліз додав операційну інфраструктуру: Docker Compose overlay для повного локального стеку, CI/CD конвеєр із 6 ітеративними hotfixes, контрактні та E2E тести, OpenAPI TypeScript-генерацію та Grafana дашборди.

Ключові цілі:

- Забезпечити повний локальний запуск усіх сервісів через Docker Compose.
- Встановити CI/CD конвеєр для автоматичної збірки, тестування та публікації Docker-образів.
- Створити тестову інфраструктуру: контрактні тести (Feign), E2E тести (Playwright).
- Додати OpenAPI-генерацію TypeScript-клієнтів для фронтенду.
- Налаштувати моніторинг через Grafana дашборди.

Б.6.2 Перелік змін

№	Ref	Категорія	Опис
1	`ecd4c7a`	[ВИПРАВЛЕНО]	Виправлення JaCoCo classDirectories finalization error на Gradle 8.14
2	`df37ce3`	[ІНШЕ]	Додано .kotlin/ до .gitignore, видалено міграційний скрипт
3	`187bd6b`	[НОВИЙ]	TypeScript client generation з OpenAPI специфікацій
4	`47b0912`	[ВИПРАВЛЕНО]	Виправлення портів сервісів, додано Gateway Dockerfile
5	`0465686`	[НОВИЙ]	Docker Compose services overlay для повного локального розгортання
6	`f5c49a0`	[НОВИЙ]	CI workflows: збірка/тестування сервісів, Docker-образи
7	`7bdc266`	[НОВИЙ]	Consumer та provider контрактні тести для міжсервісних API
8	`94b20d0`	[НОВИЙ]	Grafana: провізійовані дашборди для здоров'я сервісів, JVM, HTTP, circuit breakers
9	`24167b2`	[НОВИЙ]	E2E тести: environment, notification delivery, negative flows (Playwright)
10	`a3e5cb5`	[НОВИЙ]	Фронтенд: task plans та web service у docker-compose
11	`08ccfb9`	[ВИПРАВЛЕНО]	CI: пропуск збірки сервісів, коли вихідний код відсутній у монорепозиторії
12	`aa34a1a`	[ВИПРАВЛЕНО]	CI: додано workflow_dispatch для ручного перезапуску
13	`09740c9`	[ВИПРАВЛЕНО]	CI: динамічна матриця для уникнення помилок парсингу виразів
14	`20720f6`	[ВИПРАВЛЕНО]	CI: compact jq output для GITHUB_OUTPUT сумісності
15	`5df3fc5`	[ВИПРАВЛЕНО]	CI: перевірка наявності src у Docker build availability check
16	`1091f12`	[ВИПРАВЛЕНО]	CI: перевірка src/main замість src/ у Docker availability check
17	`8b089d5`	[НОВИЙ]	Events: MAGIC_LINK template type для magic link автентифікації

№	Ref	Категорія	Опис
18	`ebf103c`	[РЕЛІЗ]	Bump version to 1.1.0

Б.6.3 Docker Compose та сервіси

Створено `docker-compose.services.yml` — overlay поверх інфраструктурного compose, що дозволяє запускати повний стек локально одною командою.

Компонент	Деталі
Overlay підхід	Сервіси визначені окремо від інфраструктури, об'єднуються при запуску
Gateway Dockerfile	Додано Dockerfile для API-шлюзу (був відсутній у v1.0.0)
Порти	Виправлено конфлікти портів між сервісами
Web-сервіс	Next.js фронтенд додано до compose
Мережа	Усі сервіси працюють у мережі `sc-net`

Б.6.4 CI/CD конвеєр

Створено GitHub Actions workflows для автоматизації збірки та публікації. Процес налаштування вимагав 6 ітеративних hotfixes (коміти 11–16) для стабілізації:

- **service-build-test.yml** — збірка та тестування кожного сервісу при зміні його вихідного коду
- **service-docker.yml** — збірка Docker-образів та публікація до GHCR
- Динамічна матриця визначає, які сервіси змінилися, та збирає лише їх
- Додано `workflow_dispatch` для ручного перезапуску workflows

Б.6.5 Тестова інфраструктура

Версія 1.1.0 встановила три рівні тестування:

Рівень	Інструмент	Опис
Контрактні тести	Spring Cloud Contract	Consumer та provider тести для Feign-клієнтів між сервісами
E2E тести	Playwright	Наскрізні тести: environment, notification delivery, negative flows
Grafana дашборди	Provisioning	4 дашборди: service health, JVM, HTTP metrics, circuit breakers

Б.6.6 OpenAPI та фронтенд-інтеграція

Додано інструментарій для автоматичної генерації TypeScript-клієнтів з OpenAPI специфікацій сервісів. Це забезпечує типобезпечну інтеграцію фронтенду з бекендом та автоматичне виявлення контрактних змін під час компіляції.

Також у v1.1.0 додано підтримку `MAGIC_LINK` template type для нової системи автентифікації через одноразові email-посилання.

Б.7 Версія 1.2.0 — Зрілість Docker та розширення (30 березня 2026)

Б.7.1 Цілі та обсяг

Третій реліз зосереджений на операційній зрілості: багатоетапні Docker-збірки, виправлення Auth API контракту, повна переробка Docker Compose, новий Telegram Bot сервіс, сканування вразливостей Trivy та формалізація інженерних стандартів.

Ключові цілі:

- Перевести усі Dockerfile на багатоетапну збірку для зменшення розміру образів та поверхні атаки.
- Виправити семантичну помилку Auth API (POST → GET).
- Переробити Docker Compose з health-based залежностями та підтримкою нових сервісів.
- Додати Telegram Bot для операційного управління feature-прапорцями.
- Впровадити сканування вразливостей у CI/CD конвеєр.
- Формалізувати інженерні стандарти через CLAUDE.md та довідкові файли.

Б.7.2 Перелік змін

№	Ref	Категорія	Опис
1	`2b9deaf`	[ІНШЕ]	Оновлення .gitignore, видалення застарілих файлів
2	`3692b7a`	[НОВИЙ]	Переписання Dockerfile усіх сервісів на багатоетапні збірки
3	`8706dd1`	[ВИПРАВЛЕНО]	Auth API: getCurrentUser змінено з POST на GET, оновлено контрактні тести
4	`bfad1f2`	[ЗМІНЕНО]	Docker Compose: health checks, Telegram Bot, моніторинг, .env.example
5	`38ff1db`	[ЗМІНЕНО]	CI/CD: tag triggers, Trivy scanning, розширена матриця збірок
6	`211f2cb`	[НОВИЙ]	Telegram Bot Service: 79 файлів, feature flag management через Telegram
7	`ca722e2`	[НОВИЙ]	Інженерна документація: CLAUDE.md, 10 довідкових файлів, review report
8	`e6a333f`	[ІНШЕ]	Telegram buildSrc, оновлені task plans, root tooling deps
9	`7e33af9`	[РЕЛІЗ]	Bump version to 1.2.0
10	`2d2aa65`	[ВИПРАВЛЕНО]	Telegram: оновлення версії спільної бібліотеки до 1.2.0
11	`fb69309`	[ВИПРАВЛЕНО]	Repo: видалення force-committed service files, виправлення .gitignore
12	`01a3708`	[ВИПРАВЛЕНО]	Repo: E2E тести перенесено до окремого репозиторію

Б.7.3 Багатоетапні Docker-збірки

Усі сервіси переведено на триетапну Docker-збірку, що забезпечує ізоляцію етапів, мінімальний runtime-образ та правильний порядок побудови спільних бібліотек.

Етап	Назва	Базовий образ	Призначення
1	lib-builder	`eclipse-temurin:21-jdk-alpine`	Побудова спільних бібліотек (`publishToMavenLocal`)
2	service-builder	`eclipse-temurin:21-jdk-alpine`	Збірка JAR сервісу з використанням бібліотек з етапу 1
3	runtime	`eclipse-temurin:21-jre-alpine`	Мінімальний образ + `HEALTHCHECK`

Ключові рішення:

- Базовий образ змінено з `gradle:jdk21-ubi-minimal` на `eclipse-temurin:21-jdk-alpine` (збірка) та `eclipse-temurin:21-jre-alpine` (runtime)
- Стандартизований користувач `spring` з UID/GID `10000/10001` у всіх сервісах
- Додано `server/.dockerignore` для оптимізації контексту збірки
- Вбудований `HEALTHCHECK` для кожного контейнера

```
# Етап 1: Побудова спільних бібліотек
FROM eclipse-temurin:21-jdk-alpine AS lib-builder
WORKDIR /workspace
COPY libs/ libs/
COPY buildSrc/ buildSrc/
RUN ./gradlew :libs:sc-auth-starter:publishToMavenLocal

# Етап 2: Збірка сервісу
FROM eclipse-temurin:21-jdk-alpine AS service-builder
COPY --from=lib-builder /root/.m2 /root/.m2
RUN ./gradlew :services:my-service:bootJar

# Етап 3: Runtime
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -g 10001 spring && adduser -u 10000 -G spring spring
COPY --from=service-builder /workspace/build/libs/*.jar app.jar
HEALTHCHECK CMD wget -qO- http://localhost:8080/actuator/health || exit 1
```

Б.7.4 Виправлення Auth API (POST → GET)

Ендпоінт отримання поточного користувача змінено з `POST` на `GET` для дотримання REST-семантики — операція читання даних не повинна використовувати метод, що передбачає створення ресурсу.

Компонент	Зміна
`IdentityServiceClient`	`@PostMapping` → `@GetMapping` для `/api/v1/auth/user`
`ProviderContractIT`	Оновлено: тепер використовує потік magic link автентифікації
`IdentityClientContractIT` (documents)	Контрактний тест оновлено відповідно до нового методу

> **Увага:** Це ламаюча зміна (breaking change) для будь-яких зовнішніх клієнтів, що використовують `POST /api/v1/auth/user`. Усі внутрішні Feign-клієнти та контрактні тести оновлено у цьому ж релізі.

Б.7.5 Docker Compose: повна переробка

Значні зміни у конфігурації Docker Compose для підвищення надійності та підтримки нових сервісів:

- **Залежності на основі здоров'я:** `service_started` → `service_healthy` для Redis та RabbitMQ — сервіси чекають на фактичну готовність інфраструктури
- **Контекст збірки:** змінено на кореневу директорію `server/` для підтримки багатоетапних збірок з доступом до спільних бібліотек
- **JWT секрети:** volume mounts для identity-service
- **Rate limiting:** конфігурація обмеження частоти для magic link
- **Telegram Bot:** новий сервіс `telegram-bot-service` + `ngrok` + `webhook-init` у профілі `telegram`
- **Іменування змінних:** виправлено `RABBIT_*` → `RABBITMQ_*`
- **Health check шлюзу:** виправлено порт з `8080` → `8079` (management port)
- **.env.example:** створено шаблон з документацією усіх змінних оточення

Б.7.6 CI/CD: розширення та сканування вразливостей

Workflow `service-docker.yml` суттєво розширено:

Можливість	Деталі
Тригери	Додано: зміни у `libs/`, `buildSrc/`, конфігах збірки; теги `v*.*.*`
Матриця збірки	Додано: `web` (фронтенд) та `telegram-bot`
Сканування вразливостей	Trivy з вивантаженням SARIF-звітів до GitHub Security
Контекст/Dockerfile	Динамічне визначення контексту та Dockerfile для кожного типу сервісу
Таймаут	30-хвилинний ліміт виконання job
Тегування образів	Семантичне версіонування (semver) Docker-тегів

Б.7.7 Telegram Bot: новий сервіс

Повністю новий Spring Boot мікросервіс для оперативного управління feature-прапорцями через Telegram. Обсяг: 79 файлів вихідного коду з повним покриттям юніт-тестами. Підтримка української та англійської мов (i18n).

Функціональність	Опис
Webhook інтеграція	Inline-клавіатури для інтерактивної взаємодії з ботом
Feature Flags CRUD	Створення, читання, оновлення та видалення прапорців через Feign-клієнт до environment-service
RabbitMQ підписки	Сповіщення про зміни feature-прапорців через чергу повідомлень
Авторизація	Авторизація на основі chat ID
Rate Limiting	Обмеження частоти запитів для захисту від зловживань
Здоров'я	Health indicator для перевірки доступності Telegram API
Міграції БД	3 Flyway-міграції для ініціалізації схеми
i18n	Локалізація: українська (uk) та англійська (en)

Б.7.8 Моніторинг та спостережуваність

Grafana провізіонування розширено до 4 дашбордів:

Дашборд	Призначення
Services Overview	Загальний стан усіх мікросервісів, HTTP-метрики, JVM
PostgreSQL	Метрики баз даних: з'єднання, транзакції, розмір
RabbitMQ	Черги, повідомлення, швидкість обробки, споживачі
Feature Flags	Статус прапорців, історія змін, використання

Конфігурація включає автоматичне провізіонування джерел даних (Prometheus) та виявлення дашбордів при першому запуску Grafana.

Б.7.9 Інженерна документація

- **CLAUDE.md** — інженерний операційний стандарт: агенти, навички, правила, формат пропозицій змін
- **docs/claude/** — 10 довідкових файлів:
  - `architecture.md` — топологія сервісів, бібліотеки, інфраструктура
  - `testing.md` — TDD-протокол, тестова інфраструктура
  - `docker.md` — Docker-робочі процеси, відомі проблеми
  - `build.md` — команди збірки, ланцюжок залежностей
  - `agents.md`, `skills.md` — специфікації агентів та навичок
  - `interaction-standards.md`, `ios-atmosphere.md` — стандарти UI/UX
- **REVIEW_REPORT.md** — аудит з 48 знахідками (UI/UX, доступність, продуктивність, безпека)
- **Оновлені плани задач:** 19 нових файлів замість 14 старих, включаючи специфікації редизайну

Також у рамках v1.2.0 E2E тести перенесено до окремого репозиторію `selection-committee-e2e-tests` для ізоляції та спрощення CI.

Б.8 Frontend — selection-committee-web

Б.8.1 Огляд

Фронтенд-застосунок платформи FICE Selection Committee розроблений як окремий репозиторій `selection-committee-web` з гілкою за замовчуванням `develop`. За період v1.2.0 створено повнофункціональний клієнтський додаток з автентифікацією, онбордінгом, дашбордом та юридичними сторінками.

Параметр	Значення
Фреймворк	Next.js 16.2 (App Router)
Мова	TypeScript (strict mode)
UI-бібліотека	shadcn/ui + Tailwind CSS v4 (lavender theme)
Серверний стан	TanStack Query
Клієнтський стан	Zustand
Форми	React Hook Form + Zod
HTTP-клієнт	Axios з JWT-інтерсепторами
Лінтинг / форматування	Biome
Unit-тести	Vitest
E2E-тести	Playwright
Пакетний менеджер	pnpm
Комітів	12 (включно з 1 merge)
Pull Requests	#1, #2

Б.8.2 Перелік змін

№	Ref	Категорія	Опис
1	`6d2d5b1`	[ІНШЕ]	Початковий scaffolding Next.js 16
2	`59ded10`	[НОВИЙ]	Production-ready Next.js 16 frontend starter
3	`4faf411`	[НОВИЙ]	Заміна login/register на magic link автентифікацію (PR #1)
4	`fc7872b`	[ВИПРАВЛЕНО]	Phase 8: критичні виправлення безпеки, доступності та продуктивності [SEC-1..6, A11Y-1..5, PERF-1..2]
5	`9fa5dda`	[ЗМІНЕНО]	Phase 9: покращення UX, доступності, продуктивності та архітектури [UX-1..2, ARCH-2..4, A11Y-6..12, PERF-3..6, SEC-9]
6	`e67b691`	[ВИПРАВЛЕНО]	Phase 10: фінальне шліфування та оптимізація [UX-3..4, A11Y-11, ARCH-5..6]
7	`ab37894`	[ВИПРАВЛЕНО]	Центрування іконки листа у check-email card, повернення getUser до GET
8	`ad56897`	[ВИПРАВЛЕНО]	Вимкнення retry React Query для 4xx/5xx серверних помилок
9	`ab381fe`	[ВИПРАВЛЕНО]	Перенесення auth state на рівень модуля для виживання HMR
10	`26841ef`	[НОВИЙ]	Редизайн головної сторінки та онбордінгу зі split-panel layout
11	`a749816`	[НОВИЙ]	v1.2.0: покращення дашборду, юридичні сторінки, UI для feature flags
12	`bb1bf1c`	[ЗЛИТТЯ]	Merge PR #2: feature/v1.2.0-ui-updates

Б.8.3 Ключові функціональності

Magic link автентифікація

Повна заміна класичного login/register потоку на автентифікацію через одноразові email-посилання. Інтеграція з backend identity-service через Axios з JWT-інтерсепторами. Маршрутизація захищена через Next.js middleware.

Редизайн головної сторінки та онбордінгу

Впроваджено split-panel layout для головної сторінки. Триетапний онбордінг: прийняття умов (з обов'язковим переглядом обох документів), заповнення профілю, підтвердження. Адаптивний дизайн з lavender темою shadcn/ui.

Dashboard, юридичні сторінки, feature flags UI

Створено повноцінний дашборд користувача, юридичні сторінки (умови використання, політика конфіденційності) та інтерфейс управління feature flags для адміністраторів (v1.2.0).

Security hardening (Phase 8)

- Захист від XSS та CSRF
- Санітизація вхідних даних
- CSP headers конфігурація
- Безпечне зберігання токенів

Accessibility (Phases 8–10)

- WCAG-сумісність інтерфейсу
- Навігація клавіатурою для всіх інтерактивних елементів
- Підтримка screen reader (ARIA-атрибути, семантична розмітка)
- Контрастність кольорів відповідно до стандартів

Performance optimization

- Вимкнення retry React Query для 4xx/5xx помилок — уникнення зайвих запитів
- Перенесення auth state на module-level для виживання HMR
- Оптимізація бандлу та завантаження ресурсів

Б.9 Ламаючі зміни та міграція

Нижче наведено повний перелік ламаючих змін (breaking changes) через усі версії:

Версія	Зміна	Вплив	Міграція
v1.0.0	Міграція з окремих репозиторіїв у монорепозиторій	Усі сервіси змінили шляхи збірки	Оновити CI/CD та скрипти збірки. Використовувати `server/` як корінь.
v1.0.0	Спільні бібліотеки через mavenLocal	Збірка сервісів вимагає попередньої публікації бібліотек	Виконувати `./gradlew publishToMavenLocal` перед `bootJar`.
v1.1.0	Зміна портів сервісів	Клієнти зі старими портами не зможуть підключитися	Оновити конфігурацію клієнтів відповідно до docker-compose.services.yml.
v1.2.0	`POST /api/v1/auth/user` → `GET`	Зовнішні клієнти, що використовують POST, отримають 405 Method Not Allowed	Замінити `POST` на `GET` у викликах до `/api/v1/auth/user`.
v1.2.0	Dockerfile: одноетапні → багатоетапні	Зміна контексту збірки на `server/`. Старі скрипти збірки не працюватимуть.	Використовувати `docker compose build` з оновленого compose-файлу.
v1.2.0	Змінні оточення: `RABBIT_*` → `RABBITMQ_*`	Сервіси з старими назвами змінних не підключаться до RabbitMQ	Оновити `.env` відповідно до `.env.example`.
v1.2.0	E2E тести перенесено до окремого репозиторію	E2E тести більше не запускаються з основного репозиторію	Клонувати `selection-committee-e2e-tests` окремо.

Б.10 Поточна архітектура системи (post-v1.2.0)

Мікросервіси

Компонент	Технологія	Призначення	Порт
Identity Service	Spring Boot 3.x	Автентифікація, magic link, JWT, управління користувачами	8081
Admission Service	Spring Boot 3.x	Заяви абітурієнтів, конкурсний відбір	8082
Documents Service	Spring Boot 3.x	Зберігання та управління документами (MinIO)	8083
Environment Service	Spring Boot 3.x	Feature flags, конфігурація середовища	8084
Notifications Service	Spring Boot 3.x	Email-сповіщення, шаблони (magic link, тощо)	8085
Telegram Bot Service	Spring Boot 3.x	Telegram-бот для управління feature flags	8086
Gateway	Spring Cloud Gateway	API-шлюз, маршрутизація, єдина точка входу	8080
Web (Next.js)	Next.js 16+	Фронтенд, SPA, Tailwind v4, shadcn/ui	3000

Інфраструктура

Компонент	Версія	Призначення	Порт
PostgreSQL	16	Основна СУБД (окрема БД для кожного сервісу)	5432
Redis	7	Кешування, сесії	6379
RabbitMQ	3	Брокер повідомлень (events між сервісами)	5672 / 15672
MinIO	latest	S3-сумісне об'єктне сховище	9000 / 9001
Mailpit	latest	Тестовий SMTP-сервер	1025 / 8025
Zipkin	latest	Розподілене трасування	9411
Prometheus	latest	Збір метрик	9090
Grafana	latest	Візуалізація, 4 провізійовані дашборди	3001

Архітектурні принципи

- **Bounded contexts:** кожен сервіс володіє своїм доменом, базою даних та схемою. Міжсервісний доступ лише через Feign HTTP або RabbitMQ events.
- **Gateway як єдина точка входу:** порт 8080. Жоден сервіс не експонує ендпоінти напряму зовнішнім споживачам.
- **Health-based залежності:** `service_healthy` для усіх сервісів у Docker Compose.
- **Багатоетапні Docker-збірки:** 3 етапи (lib-builder → service-builder → runtime) для мінімального образу.
- **Magic link автентифікація:** без паролів, лише email-посилання через notifications-service.
- **Feature flags через Telegram:** оперативне управління без доступу до адмін-панелі.
- **Автоматичний моніторинг:** Grafana дашборди провізіонуються при першому запуску.
- **Кожен сервіс має HEALTHCHECK:** вбудований у Dockerfile + `/actuator/health` ендпоінт.
- **Фронтенд як окремий репозиторій:** `selection-committee-web` (гілка `develop`) — Next.js 16, TypeScript, shadcn/ui, незалежний цикл розробки та деплою.

Б.11 Як запускати / деплоїти / моніторити / тестувати

Б.11.1 Запуск

```
# 1. Скопіювати шаблон змінних оточення
cp infra/.env.example infra/.env

# 2. Запустити інфраструктуру (PostgreSQL, Redis, RabbitMQ, MinIO, ...)
cd infra && docker compose up -d

# 3. Запустити мікросервіси
docker compose -f docker-compose.services.yml up -d

# 4. (Опціонально) Запуск з Telegram Bot
docker compose -f docker-compose.services.yml --profile telegram up -d
```

> **Передумови:** Docker та Docker Compose мають бути встановлені. Усі сервіси запускаються у мережі `sc-net`. Перша збірка може тривати 5–10 хвилин через завантаження залежностей.

Б.11.2 Деплой

Тригер	Дія
Push до `main`	Збірка та публікація Docker-образів змінених сервісів до GHCR
Тег `v*.*.*`	Повна перезбірка усіх сервісів із семантичним тегуванням
Кожна збірка	Trivy сканує образ на вразливості, результати у GitHub Security (SARIF)

Процес: code review → CI перевірки → merge до main → автоматична збірка Docker-образів → Trivy scan → публікація до GHCR. Для релізу — git tag `v*.*.*` тригерить повну перезбірку з семантичними тегами (`latest`, `v1.2.0`, `v1.2`, `v1`).

Б.11.3 Моніторинг

Інструмент	URL	Призначення
Grafana	`http://localhost:3001`	Візуалізація метрик (логін: `admin/admin`)
Zipkin	`http://localhost:9411`	Розподілене трасування запитів між сервісами
Prometheus	`http://localhost:9090`	Збір та запити метрик
Actuator	`/actuator/health`	Перевірка здоров'я кожного сервісу
RabbitMQ UI	`http://localhost:15672`	Управління чергами та обмінниками
MinIO Console	`http://localhost:9001`	Управління об'єктним сховищем
Mailpit	`http://localhost:8025`	Перегляд тестових email-повідомлень

Б.11.4 Тестування

Рівень	Команда	Вимоги
Unit-тести	`./gradlew test`	Запускаються для кожного сервісу окремо
Integration-тести	`./gradlew integrationTest`	Потребують Docker (Testcontainers)
Contract-тести	Частина `integrationTest`	У `src/integrationTest/**/contract/`
E2E-тести	Playwright	Окремий репозиторій `selection-committee-e2e-tests`

Б.12 Детальне порівняння «Було → Стало»

1. Структура репозиторію

Було: ~~Окремі репозиторії для кожного сервісу~~

Стало: Gradle multi-module монорепозиторій

Як це працює: Усі мікросервіси, спільні бібліотеки та інфраструктурні конфігурації об'єднані у єдиний репозиторій з централізованою системою збірки. У директорії `server/buildSrc/` визначено convention plugins: `sc.java-conventions` задає базові налаштування Java-проєкту, `sc.spring-boot-service` — конфігурацію Spring Boot сервісу, `sc.testing-conventions` — тестові залежності та налаштування, `sc.jacoco-conventions` — покриття коду. Централізоване управління залежностями через version catalogs забезпечує однакові версії бібліотек у всіх модулях. Єдина точка входу `./gradlew` дозволяє збирати, тестувати та публікувати будь-який компонент.

Чому це важливо: *Єдиний репозиторій унеможливлює версійну неузгодженість між сервісами, спрощує рефакторинг крос-сервісних змін та забезпечує атомарність релізів.*

2. Docker-збірки

Було: ~~Відсутні або одноетапні Dockerfile~~

Стало: Багатоетапні 3-стадійні збірки

Як це працює: Кожен Dockerfile містить три етапи. Етап 1 (lib-builder) на базі `eclipse-temurin:21-jdk-alpine` копіює спільні бібліотеки та виконує `publishToMavenLocal`, роблячи їх доступними для наступного етапу. Етап 2 (service-builder) копіює mavenLocal з попереднього етапу та збирає JAR-файл конкретного сервісу через `bootJar`. Етап 3 (runtime) використовує мінімальний `eclipse-temurin:21-jre-alpine`, створює non-root користувача `spring` (UID 10000, GID 10001), копіює лише зібраний JAR та встановлює вбудований `HEALTHCHECK` через `wget` до `/actuator/health`.

Чому це важливо: *Фінальний образ містить лише JRE та JAR-файл, що значно зменшує поверхню атаки. Non-root користувач запобігає ескалації привілеїв всередині контейнера.*

3. CI/CD конвеєр

Було: ~~Відсутній~~ **Стало:** GitHub Actions з двома workflow

Як це працює: Workflow `service-build-test.yml` запускається при кожному Pull Request та виконує unit- та integration-тести для змінених сервісів. Workflow `service-docker.yml` активується при merge до main або створенні semver-тегу: він будує Docker-образи та публікує їх до GHCR. Динамічна матриця автоматично визначає, які сервіси змінилися (аналізуючи шляхи файлів у коміті), та збирає лише їх. Для кожного сервісу динамічно визначається контекст збірки та шлях до Dockerfile.

Чому це важливо: *Автоматизація виключає людський фактор у процесі збірки та публікації. Динамічна матриця економить час, збираючи лише змінені сервіси.*

4. Сканування вразливостей

Було: ~~Відсутнє~~ **Стало:** Trivy у CI/CD конвеєрі

Як це працює: Trivy — open-source сканер від Aqua Security, що аналізує шари контейнерного образу на наявність відомих CVE (Common Vulnerabilities and Exposures). Він запускається як крок у `service-docker.yml` після кожної збірки образу. Trivy перевіряє всі пакети операційної системи та Java-залежності у фінальному образі. Результати генеруються у форматі SARIF (Static Analysis Results Interchange Format) та автоматично завантажуються до вкладки GitHub Security. Таймаут job встановлено на 30 хвилин.

Чому це важливо: *Автоматичне сканування виявляє відомі вразливості ще до деплою, забезпечуючи безперервний контроль безпеки без ручного втручання.*

5. Автентифікація

Було: ~~Логін/пароль~~

Стало: Magic link (вхід через email-посилання)

Як це працює: Користувач вводить email-адресу на сторінці входу. Identity-service генерує JWT-токен з обмеженим терміном дії та вбудовує його у посилання. Notifications-service доставляє лист з посиланням через SMTP (у локальній розробці використовується Mailpit). Користувач натискає посилання, токен валідується на сервері, встановлюється сесійний cookie. Axios-інтерсептори на фронтенді автоматично оновлюють токен при його закінченні. Rate limiting налаштовано для ендпоінту magic link для захисту від зловживань.

Чому це важливо: *Magic link виключає ризик компрометації паролів, фішингу через підроблені форми входу та використання слабких паролів користувачами.*

6. Auth API семантика

Було: ~~POST /api/v1/auth/user для getCurrentUser~~

Стало: GET /api/v1/auth/user для getCurrentUser

Як це працює: HTTP-метод POST семантично означає "створити ресурс", тоді як операція getCurrentUser лише зчитує дані. Виправлено `@PostMapping` на `@GetMapping` у Feign-інтерфейсі `IdentityServiceClient`. Оновлено `ProviderContractIT` для відповідності потоку magic link автентифікації. Оновлено `IdentityClientContractIT` у documents-service. Видалено застарілий `EnvClientContractIT`, що більше не відповідав актуальному контракту.

Чому це важливо: *Дотримання REST-семантики забезпечує передбачуваність API, коректну роботу HTTP-кешування та сумісність із проксі-серверами.*

7. Docker Compose

Було: ~~Базовий compose без health checks~~

Стало: Health-based compose з повним стеком

Як це працює: Використовується дворівневий overlay: `docker-compose.yml` визначає інфраструктуру (PostgreSQL, Redis, RabbitMQ, MinIO, Mailpit, Zipkin, Prometheus, Grafana), а `docker-compose.services.yml` — мікросервіси. Залежності використовують `condition: service_healthy` замість `service_started`, що гарантує фактичну готовність залежного компонента. JWT-секрети монтуються через volume mounts для identity-service. Файл `.env.example` документує усі змінні оточення. Telegram-профіль (`--profile telegram`) додає ngrok + webhook-init + telegram-bot-service.

Чому це важливо: *Health-based залежності виключають стартові гонки, коли сервіс намагається підключитися до ще не готової інфраструктури. Overlay-підхід дозволяє запускати лише потрібні компоненти.*

8. Telegram-інтеграція

Було: ~~Відсутня~~

Стало: Повноцінний Telegram Bot сервіс

Як це працює: Spring Boot сервіс використовує webhook-архітектуру (не polling) — Telegram надсилає HTTP POST на ендпоінт сервісу при кожному повідомленні від користувача. Для локальної розробки ngrok створює публічний тунель, через який Telegram досягає локальний сервер. Inline-клавіатури забезпечують інтерактивний інтерфейс безпосередньо у месенджері. Сервіс комунікує з environment-service через Feign-клієнт для CRUD-операцій з feature flags. Підписка на RabbitMQ забезпечує отримання сповіщень про зміни прапорців. Авторизація на основі chat ID, per-user rate limiting, 3 Flyway-міграції, i18n: українська (uk) та англійська (en).

Чому це важливо: *Telegram-бот надає оперативний доступ до управління feature flags без необхідності відкривати адмін-панель або мати доступ до серверної інфраструктури.*

9. Моніторинг

Було: ~~Відсутній~~

Стало: Prometheus + Grafana + Zipkin

Як це працює: Спільна бібліотека `sc-observability-starter` автоматично конфігурує Micrometer для експорту метрик до Prometheus у кожному сервісі, що її підключає. Grafana має 4 автоматично провізійовані дашборди (services overview, PostgreSQL, RabbitMQ, feature flags) через JSON-файли у `infra/grafana/dashboards/`. Zipkin забезпечує розподілене трасування — кожен HTTP-запит отримує унікальний trace ID, що дозволяє відстежити його шлях через усі мікросервіси. Усе провізіонується автоматично при першому `docker compose up`.

Чому це важливо: *Моніторинг дозволяє виявляти проблеми продуктивності, деградацію здоров'я сервісів та міжсервісні затримки до того, як вони вплинуть на користувачів.*

10. Контрактні тести

Було: ~~Відсутні~~

Стало: Spring Cloud Contract (provider + consumer)

Як це працює: Provider-side тести (`ProviderContractIT`) визначають контракти через анотації та автоматично генерують HTTP-стаби для кожного ендпоінту. Consumer-side тести (`IdentityClientContractIT`) перевіряють, що Feign-клієнти відповідають контракту провайдера — тобто формат запитів та відповідей збігається. Тести розташовані у `src/integrationTest/**/contract/` та запускаються як частина `./gradlew integrationTest`. При зміні API-контракту тести автоматично виявляють розбіжності.

Чому це важливо: *Контрактні тести ловлять ламаючі зміни API до деплою, коли виправлення ще дешеве, замість виявлення проблем у production.*

11. E2E тести

Було: ~~Відсутні~~ **Стало:** Playwright у окремому репозиторії

Як це працює: Спочатку E2E тести знаходилися у монорепозиторії, але у v1.2.0 їх мігровано до окремого репозиторію `selection-committee-e2e-tests` для ізоляції та спрощення CI. Тести використовують Page Object Model (POM) — кожна сторінка представлена класом з методами для взаємодії. Покриття включає: управління середовищем, доставку сповіщень, негативні сценарії. Для виконання потрібен повний Docker-стек. Доступний запис відео для дебагу.

Чому це важливо: *E2E тести верифікують повний користувацький потік від UI до бази даних, виявляючи проблеми інтеграції, які не видно на рівні unit- або contract-тестів.*

12. OpenAPI генерація

Було: ~~Відсутня~~ **Стало:** TypeScript client generation

Як це працює: Директорія `tools/openapi-gen/` містить `config.yml` та скрипти генерації (PowerShell + Bash). Скрипти зчитують OpenAPI-специфікації із запущених сервісів (ендпоінт `/v3/api-docs`) та генерують TypeScript-типи й код API-клієнта. Згенеровані типи забезпечують типобезпечну інтеграцію фронтенду з бекендом: будь-яка зміна контракту виявляється на етапі компіляції TypeScript, а не під час виконання.

Чому це важливо: *Автоматична генерація виключає ручне дублювання типів та гарантує синхронність контракту між frontend та backend.*

13. Frontend

Було: ~~Відсутній~~ **Стало:** Next.js 16 з повним redesign

Як це працює: Застосунок побудований на App Router з розподілом на server та client компоненти. Бібліотека shadcn/ui з кастомною lavender-темою на Tailwind CSS v4 забезпечує консистентний дизайн. TanStack Query управляє серверним станом (кешування API-відповідей, автоматичний рефетчинг). Zustand використовується для клієнтського стану. React Hook Form + Zod забезпечують валідацію форм зі схемами. Biome замінює ESLint + Prettier як єдиний інструмент лінтингу та форматування. Split-panel layout для онбордінгу. Security headers (X-Frame-Options, CSP, X-Content-Type-Options) налаштовані у Next.js middleware.

Чому це важливо: *Сучасний стек забезпечує продуктивність розробки, типобезпеку, консистентний UI та захист від поширених web-вразливостей.*

14. Змінні оточення

Було: ~~RABBIT_* (нестандартний префікс)~~ **Стало:** RABBITMQ_* (стандартний префікс)

Як це працює: Префікс змінних оточення стандартизовано для узгодженості з офіційним найменуванням RabbitMQ. Створено файл `infra/.env.example` як єдине джерело істини для всіх змінних оточення системи. Файл документує кожну змінну з очікуваними значеннями та коментарями. Перед першим запуском `.env.example` копіюється у `.env` та заповнюється реальними значеннями.

Чому це важливо: *Стандартне найменування спрощує онбордінг нових розробників та зменшує ризик помилок конфігурації.*

15. Інженерна документація

Було: ~~Відсутня~~ **Стало:** CLAUDE.md + 10 довідкових файлів

Як це працює: `CLAUDE.md` — інженерний операційний стандарт, що визначає ролі агентів (Solution Architect, Backend/Frontend Engineer, QA та ін.), навички активації, операційні правила, фреймворк прийняття рішень, failure-aware мислення та формат пропозицій змін. 10 довідкових файлів у `docs/claude/` покривають архітектуру, тестування, Docker, збірку, агенти, навички, стандарти взаємодії та дизайн-систему. `REVIEW_REPORT.md` містить результати аудиту з 48 знахідками у категоріях UI/UX, доступності, продуктивності та безпеки.

Чому це важливо: *Формалізовані стандарти забезпечують консистентність інженерних рішень та прискорюють онбордінг, перетворюючи знання з "в голові розробника" на "в репозиторії".*

Б.13 Ризики та подальші кроки

Б.13.1 Відомі ризики

Ризик	Опис	Мітигація
Стартові гонки	Деякі сервіси можуть спробувати підключитися до інфраструктури до її повної ініціалізації	Health-based залежності (`service_healthy`) мінімізують ризик, але не виключають повністю
Розмір Docker-образів	Перша збірка завантажує всі Gradle залежності, що збільшує час CI	Запланувати кешування Docker-шарів у CI
E2E тести в окремому репозиторії	Розрив між кодом сервісів та E2E тестами може призвести до неузгодженості	Контрактні тести в основному репозиторії ловлять більшість порушень
Telegram Bot залежність від ngrok	Локальна розробка Telegram Bot вимагає ngrok для webhook	ngrok налаштовано у профілі `telegram`, не впливає на основний стек
Одна БД PostgreSQL	Усі сервіси використовують один екземпляр PostgreSQL з різними базами	Flyway міграції ізолюють схеми. Для production рекомендовано окремі інстанси.

Б.13.2 Подальші кроки

Пріоритет	Задача	Опис
[ВИСОКИЙ]	Тегування v1.2.0 **[ВИКОНАНО]**	Створення git-тегу `v1.2.0` для публікації Docker-образів з семантичними тегами
[ВИСОКИЙ]	Редизайн фронтенду **[В ПРОЦЕСІ]**	Виконання планів redesign-01 – redesign-07 для оновлення UI відповідно до ios-atmosphere стандарту. Split-panel redesign завершено, плани redesign-01..07 залишаються.
[СЕРЕДНІЙ]	Виправлення доступності **[В ПРОЦЕСІ]**	Реалізація знахідок з REVIEW_REPORT.md щодо accessibility. Фази 8–10 адресували основні знахідки.
[СЕРЕДНІЙ]	Кешування Docker-шарів	Оптимізація CI через кешування Gradle залежностей між збірками
[ПЛАНОВИЙ]	E2E покриття Telegram Bot	Інтеграційні тести з реальним Telegram API (тестове оточення)
[ПЛАНОВИЙ]	Production deployment guide	Документація для розгортання у production-середовищі з окремими БД та секретами

Б.14 Контроль документа

Параметр	Значення
Номер документа	FICE-REL-2026-003
Класифікація	Внутрішній документ
Статус	Затверджено
Версія документа	4.0
Дата створення	30 березня 2026 р.
Дата останньої ревізії	30 березня 2026 р.
Автор	Release Engineering Team
Охоплені версії ПЗ	v1.0.0, v1.1.0, v1.2.0
Охоплений період	25 – 30 березня 2026 р.
Наступна планова ревізія	При випуску v1.3.0 або при суттєвих архітектурних змінах

Правила оновлення: цей документ оновлюється при кожному мажорному або мінорному релізі. Патч-релізи документуються як додаток, якщо містять ламаючі зміни або суттєві виправлення.

Додаток A: Повний журнал комітів (39 backend + 12 frontend)

Backend — fice-selection-committee

39 non-merge комітів у репозиторії `fice-selection-committee/fice-selection-committee` (гілка `main`).

v1.0.0 — Фундамент (25–26 березня 2026)

№	SHA	Тип	Повідомлення
1	`71b7789`	feat	add Gradle multi-module monorepo build infrastructure
2	`8a8c205`	fix	add execute permission to gradlew
3	`6db6b73`	feat	merge sc-auth-starter into monorepo
4	`046d46b`	fix	include sc-auth-starter client package and scope gitignore paths
5	`6701695`	fix	add unauthorized entry point and normalize role authority names
6	`999dc4c`	chore	add local development startup script and project logo
7	`d6f74b1`	feat	implement shared libraries (sc-test-common, sc-observability-starter, sc-s3-starter)
8	`bdde5d4`	fix	register S3BucketInitializer as bean in auto-configuration (sc-s3-starter)
9	`30f2b86`	feat	add GitHub Packages publishing for shared libraries v1.0.0

v1.1.0 — Інфраструктура та тестування (26–28 березня 2026)

№	SHA	Тип	Повідомлення
10	`ecd4c7a`	fix	resolve jacoco classDirectories finalization error on Gradle 8.14 (buildSrc)
11	`df37ce3`	chore	add .kotlin/ to .gitignore and remove migration script
12	`187bd6b`	feat	add TypeScript client generation tooling (openapi)
13	`47b0912`	fix	correct service ports and add Gateway Dockerfile (docker)
14	`0465686`	feat	add services Docker Compose overlay for full-stack local deployment
15	`f5c49a0`	ci	add service build/test and Docker image workflows
16	`7bdc266`	test	add consumer and provider contract tests for inter-service APIs
17	`94b20d0`	feat	add provisioned dashboards for service health, JVM, HTTP, and circuit breakers (grafana)
18	`24167b2`	test	add environment, notification delivery, and negative flow E2E tests
19	`a3e5cb5`	feat	add frontend task plans and web service to docker-compose (infra)
20	`08ccfb9`	fix	skip service builds when source code is not in monorepo (ci)
21	`aa34a1a`	fix	add workflow_dispatch trigger for manual re-runs (ci)
22	`09740c9`	fix	use dynamic matrix to avoid expression parse errors (ci)
23	`20720f6`	fix	use compact jq output for GITHUB_OUTPUT compatibility (ci)
24	`5df3fc5`	fix	check for src directory in Docker build availability check (ci)
25	`1091f12`	fix	check for src/main instead of src/ in Docker availability check (ci)
26	`8b089d5`	feat	add MAGIC_LINK template type for magic link auth (events)
27	`ebf103c`	release	bump version to 1.1.0

v1.2.0 — Зрілість Docker та розширення (30 березня 2026)

№	SHA	Тип	Повідомлення
28	`2b9deaf`	chore	update .gitignore and remove obsolete files
29	`3692b7a`	feat	rewrite service Dockerfiles to multi-stage builds (docker)
30	`8706dd1`	fix	change getCurrentUser from POST to GET and update contract tests (auth)
31	`bfad1f2`	feat	overhaul docker-compose with health checks, telegram bot, and monitoring (infra)
32	`38ff1db`	feat	enhance Docker CI with tag triggers, Trivy scanning, and expanded matrix (ci)
33	`211f2cb`	feat	add telegram bot service for feature flag management (telegram)
34	`ca722e2`	docs	add engineering standards, Claude Code references, and review report
35	`e6a333f`	chore	add telegram buildSrc, updated task plans, and root tooling deps
36	`7e33af9`	release	bump version to 1.2.0
37	`2d2aa65`	fix	update shared library version to 1.2.0 (telegram)
38	`fb69309`	fix	remove force-committed service files, fix .gitignore, add release docs (repo)
39	`01a3708`	fix	move e2e tests to separate repo, add to .gitignore (repo)

Frontend — selection-committee-web (гілка `develop`)

№	SHA	Тип	Повідомлення
1	`6d2d5b1`	chore	initial Next.js 16 scaffolding
2	`59ded10`	feat	production-ready Next.js 16 frontend starter
3	`4faf411`	feat	feat(auth): replace login/register with magic link flow (#1)
4	`fc7872b`	fix	fix(security,a11y,perf): Phase 8 critical fixes [SEC-1..6, A11Y-1..5, PERF-1..2]
5	`9fa5dda`	feat	feat(ux,a11y,perf,arch): Phase 9 major improvements [UX-1..2, ARCH-2..4, A11Y-6..12, PERF-3..6, SEC-9]
6	`e67b691`	fix	fix(ux,a11y,arch): Phase 10 polish and optimization [UX-3..4, A11Y-11, ARCH-5..6]
7	`ab37894`	fix	fix(ux): center mail icon in check-email card, revert getUser to GET
8	`ad56897`	fix	fix(perf): skip React Query retry on 4xx/5xx server errors
9	`ab381fe`	fix	fix(auth): move auth state to module-level to survive HMR
10	`26841ef`	feat	feat(ui): redesign homepage and onboarding flow with split-panel layout
11	`a749816`	feat	feat(ui): v1.2.0 dashboard improvements, legal pages, feature flags UI
12	`bb1bf1c`	merge	Merge pull request #2 from fice-selection-committee/feature/v1.2.0-ui-updates